Tutorial 2: Dynamic Programming

Reinforcement Learning Course

Learning Objectives

By the end of this tutorial, you should be able to:

- Apply the Bellman equations iteratively to evaluate a policy
- Carry out policy improvement and policy iteration by hand
- Implement value iteration and extract the optimal policy
- Visualize value functions and policies in GridWorld environments

Notation Reference

- $\mathcal{S}$: State space, $s \in \mathcal{S}$: a state
- $\mathcal{A}$: Action space, $a \in \mathcal{A}$: an action
- $P(s'|s,a)$: Transition probability
- $P(s', r|s, a)$: Transition and reward probability
- $R(s, a, s')$: Reward function
- $\pi(a|s)$: Policy (probability of action a in state s)
- $V^\pi(s)$: State value function under policy π
- $Q^\pi(s, a)$: Action value function under policy π
- γ: Discount factor, $0 \leq \gamma < 1$

1 Policy Evaluation

1.1 Hand Calculation of Policy Evaluation

Consider a 2-state MDP with states $\{s_1, s_2\}$ and two actions $\{a_1, a_2\}$. The agent follows a stochastic policy and the environment has stochastic transitions.

MDP Definition:

State space: $\mathcal{S} = \{s_1, s_2\}$

Action space: $\mathcal{A} = \{a_1, a_2\}$

Transition probabilities $P(s'|s,a)$:

$$\begin{aligned} P(s_1|s_1,a_1) &= 0.8, \quad P(s_2|s_1,a_1) = 0.2 \\ P(s_1|s_1,a_2) &= 0.3, \quad P(s_2|s_1,a_2) = 0.7 \\ P(s_1|s_2,a_1) &= 0.6, \quad P(s_2|s_2,a_1) = 0.4 \\ P(s_1|s_2,a_2) &= 0.1, \quad P(s_2|s_2,a_2) = 0.9 \end{aligned}$$

Or in matrix form, where rows represent current states and columns represent next states:

$$P^{a_1} = \begin{bmatrix} 0.8 & 0.2 \\ 0.6 & 0.4 \end{bmatrix}, \quad P^{a_2} = \begin{bmatrix} 0.3 & 0.7 \\ 0.1 & 0.9 \end{bmatrix}$$

Reward function $R(s,a,s')$:

$$\begin{aligned} R(s_1,a_1,s_1) &= 2, \quad R(s_1,a_1,s_2) = 5 \\ R(s_1,a_2,s_1) &= 1, \quad R(s_1,a_2,s_2) = 3 \\ R(s_2,a_1,s_1) &= 4, \quad R(s_2,a_1,s_2) = 1 \\ R(s_2,a_2,s_1) &= 0, \quad R(s_2,a_2,s_2) = 2 \end{aligned}$$

Or in matrix form, where $R^a_{ij} = R(s_i,a,s_j)$:

$$R^{a_1} = \begin{bmatrix} 2 & 5 \\ 4 & 1 \end{bmatrix}, \quad R^{a_2} = \begin{bmatrix} 1 & 3 \\ 0 & 2 \end{bmatrix}$$

Policy $\pi(a|s)$:

$$\begin{aligned} \pi(a_1|s_1) &= 0.7, \quad \pi(a_2|s_1) = 0.3 \\ \pi(a_1|s_2) &= 0.4, \quad \pi(a_2|s_2) = 0.6 \end{aligned}$$

Discount factor: $\gamma = 0.9$

(a) Write out the full Bellman equation for $V^\pi(s_1)$ and $V^\pi(s_2)$ under this stochastic policy. Show all probability weightings.

(b) Starting from $V_0(s_1) = 0$ and $V_0(s_2) = 0$, perform 2 iterations of policy evaluation. Show all intermediate calculations.

2 Policy Iteration

2.1 Hand Calculation of Policy Iteration

Return to the 2-state MDP from Question 3.1 of Tutorial 1, with $\gamma = 0.9$:

s	a	s'	$P(s'\|s,a)$	$R(s,a,s')$
s_1	a_1	s_1	0.6	4
s_1	a_1	s_2	0.4	1
s_1	a_2	s_2	1.0	3
s_2	a_1	s_1	1.0	1
s_2	a_2	s_1	0.2	-2
s_2	a_2	s_2	0.8	0

In Tutorial 1 you showed that the policy $\pi_0(s_1) = a_1$, $\pi_0(s_2) = a_2$ has value function $V^{\pi_0}(s_1) = 10$, $V^{\pi_0}(s_2) = 5$. Starting from π_0, run policy iteration by hand:

(a) **Policy improvement:** Compute $Q^{\pi_0}(s, a)$ for all four state-action pairs and find the greedy policy $\pi_1(s) = \arg\max_a Q^{\pi_0}(s, a)$.

(b) **Policy evaluation:** Compute V^{π_1} exactly by solving the Bellman equations for π_1.

(c) Check that $V^{\pi_1}(s) \geq V^{\pi_0}(s)$ for both states. Which result guarantees this?

(d) **Policy improvement again:** Compute $Q^{\pi_1}(s, a)$ for all four state-action pairs and find the greedy policy π_2. What can you conclude, and what are V^* and π^*?

3 Implementing Value Iteration

3.1 Implementing Value Iteration

Open `tutorial_template.py`. Complete the marked sections:

(a) **Implement** `value_iteration`: Follow the algorithm:

> (a) Initialize $V(s) = 0$ for all s
>
> (b) Repeat until convergence:
>
> - For each state s: $V_{\text{new}}(s) = \max_a \sum_{s'} P(s'|s, a)[R + \gamma V(s')]$
> - If $\max_s |V_{\text{new}}(s) - V(s)| < \theta$: break

(b) **Implement** `extract_policy`: Extract optimal policy from V^*.

Visualize: Generate heatmaps and policy arrows (code provided).
Is your solution correct? If the output heatmap values and policy arrows match the figures on slide 27 of Lecture 2, you are likely correct!